

Tutorial - Semana 7 - Encontro 2

Introdução

Este encontro não introduz nenhum recurso novo da Unreal. Ele é dedicado a fechar as pontas soltas do Módulo 2: integrar, em um único fluxo jogável, tudo o que foi construído desde a Semana 4 — `GameMode`, `GameState`, `PlayerController`, `GameInstance`, `BPI_Interactable`, `Event Dispatchers`, `DT_Items` e o `BP_SaveGame / SaveComponent` do Encontro 1 — e submeter esse fluxo a dois instrumentos de avaliação que aparecem pela primeira vez na disciplina: o Code Review (Rubrica 4) e o Playtest coletivo (Rubrica 5). Este tutorial funciona como um roteiro de integração e checklist de preparação, não como uma demonstração de recurso novo.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar o processo de integração durante o laboratório e revisar os passos depois da aula.

Objetivos da Semana

- Revisar de forma integrada `GameMode`, `GameState`, `PlayerController`, `GameInstance`, `BPI_Interactable`, `Event Dispatchers`, `DT_Items` e `SaveGame`.
- Integrar os desafios acumulados do módulo (portas, alavancas, baús) em um único fluxo jogável.
- Justificar tecnicamente, em Code Review, as decisões de arquitetura adotadas.

Resultado Esperado ao Final da Semana

Um fluxo jogável único no nível de teste, integrando portas/alavancas, baús/itens e `SaveGame`, com o progresso completo persistindo corretamente entre sessões, avaliado pelo Code Review e pelo Playtest coletivo conforme as Rubricas 4 e 5 do Sistema de Avaliação.

Pré-requisitos

- Todos os sistemas do Módulo 2 (Semanas 4 a 7) implementados nos respectivos projetos de cada grupo, incluindo `BP_SaveGame / SaveComponent` do Encontro 1.
-

Antes de começar

O que você deverá possuir antes desta semana

- `BP_GameMode` , `BP_GameState` , `BP_PlayerController` e `BP_GameInstance` customizados (Semana 4).
- `BPI_Interactable` e Event Dispatcher de interação (Semana 5), aplicados a ao menos um objeto interativo (porta, alavanca ou equivalente).
- `DT_Items` tipada por `S_ItemData` / `E_ItemType` (Semana 6), aplicada a ao menos um Actor de coleta (`BP_Chest` / `BP_Pickup`).
- `BP_SaveGame` e `SaveComponent` funcionais (Encontro 1 desta semana).

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo é necessário; o encontro reutiliza integralmente os assets já presentes no projeto.

Projeto esperado

O projeto de cada grupo com todos os sistemas do Módulo 2 implementados, ainda que testados de forma isolada entre si — a integração completa é justamente o objetivo deste encontro.

Parte 1 — Revisão integrada do Módulo 2

Objetivo

Revisar, em conjunto com a turma, os sistemas construídos entre as Semanas 4 e 7, verificando que cada um está presente e funcional antes de iniciar a integração.

Conceito

Este encontro não introduz um conceito universal novo — ele consolida a ideia que atravessou todo o Módulo 2: um Vertical Slice funcional não é a soma de sistemas isolados, mas a integração coerente entre eles. GameMode e GameState fornecem o contexto de partida; PlayerController e GameInstance fazem a ponte entre jogador e persistência entre níveis; BPI_Interactable e Event Dispatchers desacoplam a comunicação entre Player e objetos do mundo; DT_Items separa dado de lógica; e o SaveGame garante que tudo isso sobreviva entre sessões. Antes de integrar, é preciso confirmar que cada peça, isoladamente, ainda está de pé.

Passo a passo

1. Abrir o projeto do grupo e listar, junto com o professor, os sistemas esperados até este ponto: BP_GameMode , BP_GameState , BP_PlayerController , BP_GameInstance , BPI_Interactable , Event Dispatcher de interação, DT_Items , BP_SaveGame e SaveComponent .
2. Para cada sistema da lista, abrir rapidamente o Blueprint correspondente e confirmar que ele compila sem erros (ícone de compilação sem avisos vermelhos).
3. Testar isoladamente, em modo Play, cada objeto interativo já implementado (porta, alavanca, baú/pickup), confirmando que cada um ainda reage corretamente à interação.
4. Anotar, em conjunto com o grupo, qualquer sistema que não esteja mais funcionando desde a última vez em que foi testado (regressões acontecem ao longo do semestre e devem ser tratadas antes da integração).

Resultado esperado

Confirmação de que todos os sistemas do Módulo 2 compilam e funcionam isoladamente, antes de iniciar a integração da Parte 2.

Verificando

Cada sistema testado em modo Play, isoladamente, sem erros de compilação e sem comportamento inesperado.

Problemas comuns

- **Assumir que um sistema "ainda funciona" sem testar de fato:** regressões silenciosas (por exemplo, uma variável renomeada em uma Data Table que quebra uma referência) só aparecem quando o sistema é efetivamente testado, não apenas revisado visualmente.

- **Pular a revisão de um sistema por falta de tempo:** um sistema não revisado que falhar durante a integração da Parte 2 é mais difícil de depurar, pois passa a competir com o restante do fluxo integrado.

Boas práticas

Trate esta revisão como o próprio grupo trataria um "smoke test" antes de uma entrega em um estúdio profissional: rápido, mas cobrindo todos os sistemas críticos, antes de qualquer trabalho adicional em cima deles.

Comparação com Unity

Não há novo recurso da Unreal a comparar nesta etapa. A discussão comparativa retoma, de forma breve, o quadro já construído nas Semanas 4 a 7: a ausência de um equivalente direto a `GameMode/GameState` na Unity (resolvido por convenção de Managers/Singletons), Interfaces em C# para `BPI_Interactable`, `UnityEvent/Actions` para Event Dispatchers, `ScriptableObject` para Data Table, e `PlayerPrefs/JSON` para `SaveGame`.

Parte 2 — Integração em um único fluxo jogável

Objetivo

Conectar, em um único percurso jogável no nível de teste, os objetos interativos, a coleta de itens e o `SaveGame`, de modo que o progresso completo seja gravado e recuperado de forma consistente.

Conceito

Cada sistema construído nas Semanas 4 a 7 foi ensinado e testado de forma relativamente isolada. A integração é o momento em que essas peças precisam coexistir no mesmo nível, na mesma sessão de jogo, sem que a comunicação de um sistema interfira incorretamente em outro. É comum que conflitos de integração só apareçam neste momento — por exemplo, o `SaveGame` não reconhecendo o estado de um objeto interativo específico, ou dois sistemas competindo pela mesma referência ao `Player`. Tratar esses conflitos como parte esperada do processo, e não como falha, é a postura correta neste encontro.

Passo a passo

1. No nível de teste, organizar em um único percurso navegável os objetos interativos já existentes: a porta ou alavanca da Semana 5, os baús/itens da Semana 6, e os checkpoints, caso já modelados.
2. Confirmar que cada objeto interativo do percurso implementa `BPI_Interactable` e dispara corretamente o Event Dispatcher correspondente.
3. Para cada Actor de coleta do percurso, confirmar que a coleta aciona `SalvarProgresso` no `SaveComponent` do Player (Encontro 1), gravando o identificador do item em `BP_SaveGame`.
4. Confirmar que o carregamento do nível aciona `CarregarProgresso` no `SaveComponent`, reaplicando corretamente o estado de todos os objetos já coletados/ativados no percurso — não apenas de um único item isolado testado no Encontro 1.
5. Testar o percurso completo, do início ao fim, em uma única sessão de Play: interagir com todos os objetos na ordem esperada, coletar todos os itens, e confirmar que o progresso é gravado.
6. Fechar o Play, reabrir o nível, e confirmar que o percurso completo é recuperado corretamente — todos os objetos já interagidos/coletados devem refletir esse estado.
7. Caso algum conflito apareça (por exemplo, um objeto específico não gravando corretamente), isolar qual sistema não está se comunicando corretamente e revisar se a comunicação passa pelos padrões corretos (Interface/Event Dispatcher/SaveComponent) antes de qualquer ajuste emergencial.
8. Documentar, junto ao grupo, qualquer pendência que não seja resolvida a tempo do Code Review, para apresentação transparente durante a avaliação.

Resultado esperado

Um fluxo jogável único no nível de teste, com todos os objetos interativos, a coleta de itens e o `SaveGame` integrados, e o progresso completo persistindo corretamente entre sessões.

Verificando

Percorrer o fluxo completo do início ao fim em uma sessão de Play, fechar e reabrir o nível, e confirmar que o estado de todos os sistemas (não apenas de um item isolado) é recuperado corretamente na nova sessão.

Problemas comuns

- **Sistemas funcionais isoladamente, mas nunca testados em conjunto:** este é o problema mais esperado deste encontro. A integração frequentemente revela que o `SaveGame` não

reconhece o estado de um objeto interativo específico, porque esse objeto nunca foi conectado ao `SaveComponent` durante sua implementação original.

- **Tratar conflitos de integração como motivo de retrabalho total:** o caminho correto é isolar o ponto exato de falha, verificar se a comunicação segue os padrões corretos (Interface/Event Dispatcher/SaveComponent), e corrigir apenas esse ponto — não recriar o sistema do zero.
- **Grupos que não concluem a integração completa a tempo:** devem apresentar o fluxo parcial no Code Review, registrando as pendências para acompanhamento nos módulos seguintes, sem que isso impeça o registro do progresso já alcançado.

Boas práticas

Utilize Comment Boxes nos pontos de integração entre sistemas de módulos diferentes (por exemplo, onde o Event Dispatcher de um baú aciona o `SaveComponent`), facilitando a leitura do fluxo completo durante o Code Review.

Comparação com Unity

Não há novo recurso da Unreal a comparar nesta etapa. O objetivo aqui não é aprofundar cada comparação já feita nas semanas anteriores, mas reforçar que a integração entre múltiplos sistemas desacoplados — não apenas cada peça isoladamente — é o que diferencia um protótipo de um gameplay funcional, princípio válido em qualquer engine.

Parte 3 — Code Review e Playtest coletivo

Objetivo

Submeter o fluxo integrado aos dois instrumentos de encerramento do Módulo 2: o Code Review (avaliação de "caixa aberta", sob a perspectiva de boas práticas) e o Playtest coletivo (avaliação sob a perspectiva da experiência do jogador).

Conceito

Um Vertical Slice funcional precisa ser avaliado sob duas perspectivas complementares. O **Code Review** verifica a qualidade da implementação por dentro: organização, nomenclatura, modularidade, reutilização de sistemas anteriores e comunicação desacoplada entre eles, conforme a Rubrica 4 do Sistema de Avaliação. O **Playtest** verifica a mesma integração pela perspectiva de quem joga sem

conhecer o projeto por dentro: funcionamento, usabilidade, bugs, feedback visual e clareza de interface, conforme a Rubrica 5. As duas faces juntas — "o sistema está bem construído" e "o sistema funciona bem para quem joga" — são exatamente o que qualquer estúdio profissional avalia antes de considerar um módulo encerrado.

Passo a passo

1. Preparar o projeto para o Code Review: garantir que os Blueprints envolvidos na integração estão organizados, comentados e sem lógica duplicada visível.
2. Durante o Code Review, abrir os Blueprints ao vivo junto ao professor e explicar, com o grupo, por que cada decisão de arquitetura foi tomada (por exemplo: por que um Actor usa Interface em vez de referência direta; por que um dado está em `DT_Items` e não hardcoded).
3. Preencher, com o professor, o instrumento de Code Review (Rubrica 4 do Sistema de Avaliação), cobrindo organização, nomenclatura, modularidade, reutilização e comunicação entre sistemas.
4. Organizar o rodízio de Playtest: identificar um jogador externo ao próprio grupo (idealmente um colega de outro grupo) para testar o fluxo integrado.
5. Durante o Playtest, o jogador externo testa o percurso completo sem explicação prévia além do mínimo necessário, enquanto o grupo observa e evita intervir, exceto em casos de bloqueio real.
6. Preencher, com o professor, o instrumento de Playtest (Rubrica 5 do Sistema de Avaliação), registrando funcionamento, usabilidade, bugs, feedback visual e clareza de interface observados durante a sessão.
7. Registrar, ao final de ambos os instrumentos, as pendências identificadas para acompanhamento nos módulos seguintes.

Resultado esperado

Code Review e Playtest coletivo concluídos, com os instrumentos das Rubricas 4 e 5 preenchidos, e pendências registradas para os módulos seguintes, quando houver.

Verificando

Instrumentos de Code Review e Playtest preenchidos e assinados pelo professor, com pendências (se houver) claramente registradas.

Problemas comuns

- **A própria equipe jogar o Playtest em vez de observar um jogador externo:** isso invalida o instrumento, pois confunde "o jogo funciona para quem o desenvolveu" com "o jogo funciona

para um jogador novo".

- **Justificar decisões técnicas apenas com "porque funcionou", sem relação com o conceito estudado:** o Code Review espera que o grupo relacione a decisão ao conceito universal por trás dela (por exemplo, desacoplamento via Interface), não apenas ao resultado funcional.
- **Ignorar problemas de clareza de interface por já conhecer o funcionamento internamente:** o grupo tende a subestimar a confusão de um jogador novo justamente por já dominar o próprio projeto.

Boas práticas

Distribua a explicação do Code Review entre os integrantes do grupo, evitando que apenas uma pessoa domine e explique todo o projeto — essa distribuição já antecipa a exigência da Rubrica 6 (Apresentações), aplicada em módulos futuros.

Comparação com Unity

Não há novo recurso da Unreal a comparar nesta etapa. Code Review e Playtest são práticas de processo de desenvolvimento, não recursos específicos de uma engine — aplicam-se da mesma forma a um projeto construído em Unity, Godot ou qualquer outro motor.

Ao final da semana

Ao final da Semana 7, cada grupo deve possuir um gameplay funcional consolidado: `GameMode`, `GameState`, `PlayerController` e `GameInstance` como framework de base; `BPI_Interactable` e `Event Dispatchers` permitindo comunicação desacoplada entre `Player` e objetos do mundo; `DT_Items` (com `S_ItemData` e `E_ItemType`) como dado de design centralizado; e `BP_SaveGame / SaveComponent` persistindo o progresso de coleta entre sessões — tudo integrado em um único fluxo jogável testado via Code Review e Playtest coletivo. Conceitualmente, a turma deve dominar a distinção entre regras de partida, estado compartilhado, comunicação desacoplada, dado de design e persistência como cinco problemas complementares de arquitetura de motores, todos resolvidos de forma integrada — não isolada — em um mesmo projeto. Este resultado encerra a Unidade II — Construir Sistemas, conforme o roadmap de `PROJECT_ARCHITECTURE.md` (seção 6).

Desafio

Cada grupo apresenta sua integração completa do Módulo 2 — todos os objetos interativos, coleta de itens e `SaveGame` funcionando em um único fluxo — justificando tecnicamente, durante o Code

Review, as escolhas de arquitetura adotadas (uso de Interfaces, Event Dispatchers, Data Table, e centralização da persistência no `SaveComponent`).

Checklist

- ☐ Todos os sistemas do Módulo 2 revisados e compilando sem erros
- ☐ Objetos interativos (porta/alavanca, baú/pickup, checkpoint quando existente) integrados em um único percurso jogável
- ☐ SaveComponent gravando corretamente o progresso de todos os objetos do percurso, não apenas de um item isolado
- ☐ Progresso completo recuperado corretamente após fechar e reabrir o nível
- ☐ Code Review realizado, com Rubrica 4 preenchida e decisões de arquitetura justificadas pelo grupo
- ☐ Playtest coletivo realizado com jogador externo ao grupo, com Rubrica 5 preenchida
- ☐ Pendências (se houver) registradas para acompanhamento nos módulos seguintes

Glossário

- **Integração:** processo de conectar sistemas construídos de forma isolada em um único fluxo funcional coerente.
- **Code Review:** instrumento avaliativo que verifica a qualidade da implementação por dentro (organização, nomenclatura, modularidade, reutilização, comunicação desacoplada), conforme Rubrica 4.
- **Playtest:** instrumento avaliativo que verifica a experiência de um jogador externo ao grupo, testando o fluxo jogável sem conhecimento prévio do projeto por dentro, conforme Rubrica 5.
- **Regressão:** perda de funcionalidade em um sistema que antes funcionava corretamente, geralmente causada por uma mudança não testada em outro ponto do projeto.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine 5 Documentation** — Saving and Loading Your Game. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/saving-and-loading-your-game-in-unreal-engine>.

- EPIC GAMES. **Unreal Engine Learning Library**. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual**, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeos sugeridos (apenas como apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**.

Imagem sugerida

Objetivo: mostrar a diferença de perspectiva entre Code Review e Playtest sobre o mesmo Vertical Slice integrado. Enquadramento: diagrama de duas lentes apontando para o mesmo objeto central. Elementos importantes: centro — ícone representando o Vertical Slice integrado (percurso com porta, baú e checkpoint); lente esquerda rotulada "Code Review — avaliação por dentro (arquitetura, nomenclatura, boas práticas)"; lente direita rotulada "Playtest — avaliação por fora (funcionamento, usabilidade, clareza)". O que deve ser destacado: as duas lentes observam o mesmo objeto, mas em perspectivas complementares, nunca substitutas uma da outra. Legenda sugerida: "Duas lentes, um só projeto: como o Módulo 2 é avaliado por dentro e por fora."